

Computer Security Exam

Professors S. Zanero & M. Carminati & A. Barenghi

09/01/2024

Last (family) Name _____

First (given) Name _____

Matricola _____

Codice Persona

Professor: [] Zanero [] Carminati [] Barenghi

Did you participate in the extra points practical test? ☐ Yes ☐ No

Do you want to withdraw? ☐ Yes ☐ No

Priority Grading [] Yes, Motivation:

[] No

Instructions

- **Duration:** 2.5 hours.
- The exam is composed of 15 pages. Check that you have all of them.
- Just as a cross-check, tell us whether you have completed challenges by appropriately putting an "X" mark.
- The exam is "closed book". The students will not be allowed to consult any **course material** and **notes**.
- **No extra devices** (e.g., phones, iPad) are **allowed**. You will be expelled if, at any time, you do not follow this rule.
- Shut down and store any electronic device. They will be subject to inspection if found and you may be expelled if you are found using one.
- You are not allowed to communicate with other students, and you will be expelled from the exam if you do.
- Please answer within the allowed space. Schemes are good; short answers are recommended.
- You can write in pen or pencil, any color, but avoid writing in red.
- No extra paper is allowed.
- **Always motivate your answer.**
- **If your final grade is below 12 (not considering the challenges), you will not be allowed to take the next exam call (even in the next session).**

- **DISTANCE MODE ONLY**

- On the computer, you will be allowed to open only the MS forms browser tab and the exam pdf. Any other application or document is prohibited.
- You will have to share your screen, leave your microphone open, and your webcam must frame both you and your workspace.
- You are responsible for having all the necessary technological equipment for the correct delivery of the exam.
- If you disable the screen sharing or the audio, you will be expelled from the virtual room and your exam will be voided.
- We will ask remote students to sustain a brief oral exam to confirm the evaluation of the written exams. This could be done for all students or for a subset, at the instructor's decision.
- Regarding the submission:
 - At the end of the exam, you will be given 10 minutes only to scan your documents and prepare to upload them.
 - After 10 minutes, you will be called by name and required to click on the submit button of the MS forms under the supervision of the supervisor of the virtual room. Any submission that will not follow this procedure (e.g., you have already submitted before being called or you are not ready to submit when called) will be voided.
 - We are not responsible for any technical issue in the submission (e.g., the exam pictures are not readable, the pdf is corrupted). We will evaluate only the uploaded readable documents.
 - We will evaluate only the uploaded **readable** documents.
 - USE YOUR PERSONAL CODE (CODICE PERSONA) ONLY AS THE NAME OF THE DOCUMENT (e.g., 12345678.pdf).

PLEASE, USE THE FORMATTING SPECIFIED IN THE QUESTION

READ CAREFULLY ALL THE POINTS OF EACH QUESTION BEFORE WRITING YOUR ANSWER

Exercise 1 - Introduction to Computer Security & Authentication (4 points)



Consider ad-blocking browser extensions designed to enhance web browsing by eliminating unwanted ads. These client-side programs use a combination of algorithms, filters, and user-defined settings to identify and block various forms of advertisements. They operate by analyzing the content of web pages and distinguishing ads from the main content, thus preventing them from displaying. Users can customize these extensions to allow non-intrusive ads or to block all ads indiscriminately. These extensions require minimal setup and are usually connected to regularly updated online public databases, where they retrieve the latest filters and rules. Many advanced versions offer additional features through integration with browser functionalities, allowing users to manage their preferences, view statistics on blocked content, or even report unblocked ads.

[2 points] 1. Suggest two relevant assets at risk in such a scenario. Motivate your answer.

Asset	Motivation
User Privacy and Data Security	Ad-blocking extensions often require access to all data on websites visited by users to effectively block ads. This level of access can put user privacy at risk if the extension is not from a trustworthy source. Malicious or poorly designed extensions could potentially track user browsing habits, collect sensitive data such as passwords, or even modify website content in harmful ways.
System Security and Performance	Browser extensions, including those for ad-blocking, have the potential to impact the overall system's integrity and performance. A poorly coded or malicious ad-blocking extension can consume excessive system resources, leading to slower computer performance or browser crashes. In more severe cases, such extensions might introduce malware or adware, compromising the integrity of the user's system.

[2 points] 2. Suggest the most likely potential threats and threat agents against such infrastructure and their operating companies.

Threats	Threat Agents
Data Theft/Malware Distribution Motivation: [...]	Cybercriminals Motivation: [...]
Surveillance and Spying/Censorship Motivation: [...]	State-Sponsored Actors Motivation: [...]

Exercise 2 - Introduction to Cryptography (6 points)

[3 points] 1. Consider a password storage scheme where the user-supplied password is hashed with a 4 bytes long, uniformly randomly sampled string (salt), employing SHA-2-256 and its digest is stored on disk, together with the random string.

Computing a single SHA-2-256 costs around 133 microwatts with current technology, and takes 4 microseconds.

What is the probability that an attacker, with a 100 kWh budget is able to guess, through bruteforcing, one randomly picked, 7 lowercase latin alphabet characters password, given digest and salt? Assume for simplicity that the entire energy budget is used for SHA-2-256 computations, without overheads.

[Solution] Investing 100 kWh ($3.6 \cdot 10^{11}$ Ws) in computing SHA-2-256 yields $3.6 \cdot 10^{11} / (133 \cdot 10^{-6} \cdot 4 \cdot 10^{-6}) = 1.915 \cdot 10^{23} \approx 2^{77}$ SHA-2-256 computation attempts

One guessing attempt at a random, 7 characters password with a 32 bit salt has a $1/(26^7 \cdot 2^{32}) \approx 1/(2^{64})$ success probability.

The energy budget of the attacker allows her to perform approximately 8000 times the amount of attempts required, on average, to guess the password, making the success practically certain.

[3 Points] 2. Consider the same password hashing strategy as before. Quantify the change in the attacker success probability if the digest of the SHA-2-256 hash digest is cut down to 160 bits, discarding the last 96 bits obtained from the computation, reducing the resistance to collisions for SHA-2-256.

[Solution] There is no change in the success probability of the attacker: The collision resistance of the hashing mechanism is reduced to a computational effort of about 2^{80} SHA-2-256 computations. The attacker is still better off in trying to search for the password (the hash domain is way smaller than its codomain, even after the cut).

Exercise 3 - Binary Vulnerabilities (6 points)

```
00 #include <stdio.h>
01 #include <stdlib.h>
02 #include <string.h>
03 #include <unistd.h>
04
05 typedef struct {
06     char username[32];
07     char secret[8];
08     char msg1[16];
09     char *msg;
10     char token[16];
11 } UserData;
12
13 int read_file(char *filename, UserData *data) {
14     FILE *file = fopen(filename, "r");
15     fscanf(file, "%7s", data->secret);
16     fclose(file);
17     printf(data->username);
18     return 0;}
19
20 void guess_token(UserData *data) {
21     char guess[16];
22
23     scanf("%15s", guess);
24     if (strcmp(data->token, guess) == 0) {
25         system("/bin/sh");
26     } else {
27         printf("Incorrect token\n");
28     }
29 }
30
31 int main() {
32     UserData data = { .msg1 = "Token generated" }; // Initialize msg1
33     clearenv(); // Clear environment variables
34
35     scanf("%31s", data.username);
36     read_file("/etc/secret", &data);
37     strcpy(data.token, data.secret);
38     strcat(data.token, data.username);
39     data.msg = data.msg1;
40     printf("%s\n", data.msg);
41     guess_token(&data);
42     return 0;
43 }
```

Assume that:

- Environment variables are located in the highest addresses.
- The program is compiled for the **x86 architecture** (32 bit) and for an environment that adopts the usual **cdecl** calling convention. Furthermore, assume that **no compiler-level or OS-level mitigations** against the exploitation of memory corruption errors are present (unless specified otherwise).
- **char * strcat(char *restrict s1, const char *restrict s2)** appends a copy of the null-terminated string *s2* to the end of the null-terminated string *s1*, then adds a terminating `'\0'`. The string *s1* must have sufficient space to hold the result.

[1 point] 1. The program is affected by typical buffer overflow and format string vulnerabilities. Complete the following table, focusing on a vulnerability per row.

Vulnerability	Line	Motivate and Modify the line to solve the detected vulnerability
Buffer Overflow	38 [0.25]	See Slides [0.25]
Format String	17 [0.25]	See Slides [0.25]

[2 points] 2. **Focus only on the stack-based buffer overflow(s) you found.** Write an exploit for this vulnerability that **must execute the following shellcode**, composed of 8 bytes, which opens a shell: **0x10 0x20 0x30 0x40 0x50 0x60 0x70 0x80**.

2.a) Describe **all the steps and assumptions** required to successfully exploit the vulnerability. Include also any assumption on how you must call and run the program: e.g., the values for the command-line arguments required to trigger the exploit correctly and/or environment variables (if any), the input provided during the execution, if multiple executions are necessary.

You can exploit the buffer overflow on the variable "token" The overflow happens with the strcat at line 38, combined with the fact that the variable username can be filled with 32 chars (31 user defined + null terminator) by scanf. This way, the strcat will copy both secret and username inside token, causing the overflow. We can fill username with a NOPsled and the shellcode and put the desired return address to overwrite the sEIP.

username = "\x90\x90\x90" ... + shellcode + target_address

2.b) Make sure that you show how the exploit will appear in the process memory with respect to the stack layout right **before and after the execution of the vulnerable line** during the program exploitation showing:

- Direction of growth and high-low addresses;
- The name of each allocated variable;
- The content of relevant registers (i.e., EBP, ESP);
- The functions stack frames.

Show also the content of the caller frame.

H	address	BEFORE	Content	AFTER
		...		
		sEIP		~address A
	EBP->	sEBP		0x50 0x60 0x70 0x80
		token []		0x10 0x20 0x30 0x40
		token []		0x90 0x90 0x90 0x90
		token []	et!	et!
		token [0-3]	secr	secr
		*msg		
		msg1 []	ted	ted
		msg1 []	nera	nera
		msg1 []	n ge	n ge
		msg1 [0-3]	toke	toke
		secret[]	et!	et!
		secret[0-3]	secr	secr
		username		
		username		
		username		
		username		
		username	~ESP	~address A
		username	0x50 0x60 0x70 0x80	0x50 0x60 0x70 0x80
		username	0x10 0x20 0x30 0x40	0x10 0x20 0x30 0x40
	ESP->	username	0x90 0x90 0x90 0x90	0x90 0x90 0x90 0x90
V				
L				

[3 points] 3. **Focus only on the format-string vulnerability you found.** Consider ONLY the correctly implemented “NX” mitigation technique. Someone told us that you can use this vulnerability to obtain a shell, even with the NX mitigation enabled. Rumors say there might be **three working exploits** that do not require neither ret2libc nor ROP. If the rumors are true, explain the exploits, otherwise explain why the vulnerability cannot be exploited. Describe **all the steps and assumptions** required to successfully execute the exploits.

1) We could use the format string to overwrite the return address of `read_file` with the address (in `.text`) of the code that prepares the call to `system("/bin/sh")`. Being the address in `.text`, we will not have issues with NX. [1 pt].

2) We could use the format string to overwrite the content of `secret` with a string and just input it into guess with the remaining chars of the username. This way we will pass the check! [1 pt].

3) We could use the format string to overwrite the `*msg` to print `secret` and just input it into guess with the remaining chars of the username. This way we will pass the check! [1 pt]

Similarly, in all cases, we need to assume ASLR disabled or we will not know the stack addresses (addr:s of ret address and user.secret, respectively).

Exercise 4 - Web Vulnerabilities (6 points)

Threats is the new social network that makes Computer Security students go crazy! Everyone is talking about it. Rumors are that the social network is not secure and that evil students are exploiting it, every day. You have been called as a penetration tester to identify and debug the issues. The website is at "<https://threats.com>", we wish you good luck.

The relevant code of the application is the following:

```
00 def is_xss_free(_string_):
01     if ("script" in _string_) or ("SCRIPT" in _string_):
02         return False
03     if ("img" in _string_) or ("IMG" in _string_):
04         return False
05     return True

06 @app.route('/profile', methods = ['GET'])
07 def profile(request):
08     if not is_session_valid(session):
09         return redirect(url_for("login"))
10
11     # Search the user in db
12     handle = request.args.get('handle')
13     user_query = "SELECT user_id, img_path FROM user WHERE handle = ?;"
14     user = db_execute_query_with_statement(user_query, handle)
15
16     # Check if the user is not present in the db
17     if user == []:
18         if is_xss_free(handle):
19             return "No user handle \"" + handle + "\" was found."
20         else:
21             return "Threat detected."
22
23     # Else, retrieve its profile image
24     img_path = user[0]["img_path"]
25     return render_template_with_image("profile.html", handle=handle, img_path=img_path)

26 @app.route('/upload_image', methods = ['POST'])
27 def upload_image():
28     if not is_session_valid(session):
29         return redirect(url_for("login"))
30
31     user_id = session["user_id"]
32     handle = session["handle"]
33
34     if 'image' not in request.files:
35         return "You need to attach a jpeg image."
36
37     image = request.files['image']
38
39     img_path = "/images/" + user_id + image.filename
40     image.save(img_path)
41
42     update_query = "UPDATE user SET img_path = ? WHERE user_id = ?;"
43     db_execute_query_with_statement(update_query, img_path, user_id)
44     return render_template_with_image("profile.html", handle=handle, img_path=img_path)
```



```
45 ## Some javascript will prevent attackers from tampering with the image upload procedure
46 ...
47 <script>
48 document.getElementById('uploadForm').addEventListener('submit', function(event) {
49     var fileInput = document.getElementById('image');
50     var fileName = 'profile_image'; // Set the filename to "profile image"
51     var allowedFormats = ['.jpg', '.jpeg'];
52
53     // Check for allowed file format
54     var validFormat = false;
55     for (var i = 0; i < allowedFormats.length; i++) {
56         if (filePath.toLowerCase().endsWith(allowedFormats[i])) {
57             validFormat = true;
58             break;
59         }
60     }
61
62     if (!validFormat) {
63         alert('Invalid file format. Only JPEG images are allowed.');
```

The relevant portion of the relational database schema used by the website is the following:

Table: **USER**

user_id (Primary Key) (Integer) (Auto increment)	handle (String) (Unique)	password (String)	img_path (String)
--	-----------------------------	----------------------	----------------------

Assume the following:

- The session is correctly handled (i.e., you cannot forge arbitrary cookies).
- The **registration** function allows only for alphanumeric user handles and is implemented correctly.
- The function **is_session_valid** correctly checks if the user is logged in.
- The function **db_execute_query_with_statement** correctly executes queries with prepared statements.
- The function **render_template** generates an html page with optional parameters that are placed in the html code. **It does not perform any sanitization of the input.**
- The **DBMS** does not support stacked queries (e.g., "SELECT ... ; INSERT INTO ... ;" fails if executed)

More details:

- The function **url_for** generates an url of the provided page of the same domain. Optional query parameters of the url can be specified as parameters.
- The function **db_execute_query_with_statement** returns a list of rows as a result, each of which has the fields specified by the **SELECT** statement. For instance, `result[4]["comment"]` is accessing the field "comment" of the 5th row.

[3 points] 1. Fill out the following table by answering the questions for each class of vulnerabilities. While answering the questions, be **exhaustive, concise, and straight to the point**. If you believe there might be a vulnerability, specify all the corresponding line/s of code.

<i>Vulnerability class</i>	<i>Is there a vulnerability of this class in the code above? How could an adversary exploit it?</i>	<i>If the vulnerability is present, explain the simplest procedure to remove it.</i>
SQL Injection (SQL-I)	<i>No, the snippet contains only prepared statements.</i>	--
cross-site scripting (XSS) Specify if reflected, stored, DOM...	<i>There is a reflected XSS in profile, line 19. The attacker could use the handle.</i>	<i>E.g., Escaping, see slides</i>
Cross-site request forgery (CSRF)	<i>Yes, an attacker could lead a victim to change their own profile picture with an evil website and a carefully crafted form.</i>	<i>E.g., CSRF tokens, see slides</i>

[1.5 points] 2. One user managed to create a web page on Threat's domain that shows the message "Alert: You have been hacked!". Can you reproduce the exploit? Explain all the steps you would take to reproduce the exploit, including the final code you would write and why it works.

The exploit is a reflected XSS in the profile page

The attacker could craft a url like this

`https://threats.com/profile?handle=<SRipt>alert("Alert: You have been hacked!")</SRipt>`

We bypass the filter as it only checks for script or SCRIPT...

[1.5 points] 3. Some students are reporting issues with their profile image. Every now and then, their image becomes a black and white flag with a skull on top. Moreover, the image is not static and the flag is waving! It looks like the new image is a gif, instead of a jpeg. We never programmed something like this to happen, some attacker must be tampering with Threats.

1. Identify the vulnerability(ies) that enable the attack and the necessary assumptions, if any.
2. Explain all the steps you would take to reproduce the exploit, targeting **@TEST_ACCOUNT**, without relying on user interaction.
3. Suggest the best way to fix the problems (i.e., avoid allowing a user to change someone else's image; avoid saving a gif instead of a jpeg).

1. *A path traversal in the image upload function (server-side). Client-side checks on the image file format. CSRF could also be a way.*
2. *Tamper with javascript client-side on the browser (e.g., redefine the "check" function) and upload an image called `../TEST_ACCOUNT/profile_image`. If the victim user always uploaded the image via the website, its image will be in `/images/TEST_ACCOUNT/profile_image`, so you will overwrite their image*
3. *Move all the checks for path traversal and image format server-side. Implement CSRF tokens.*

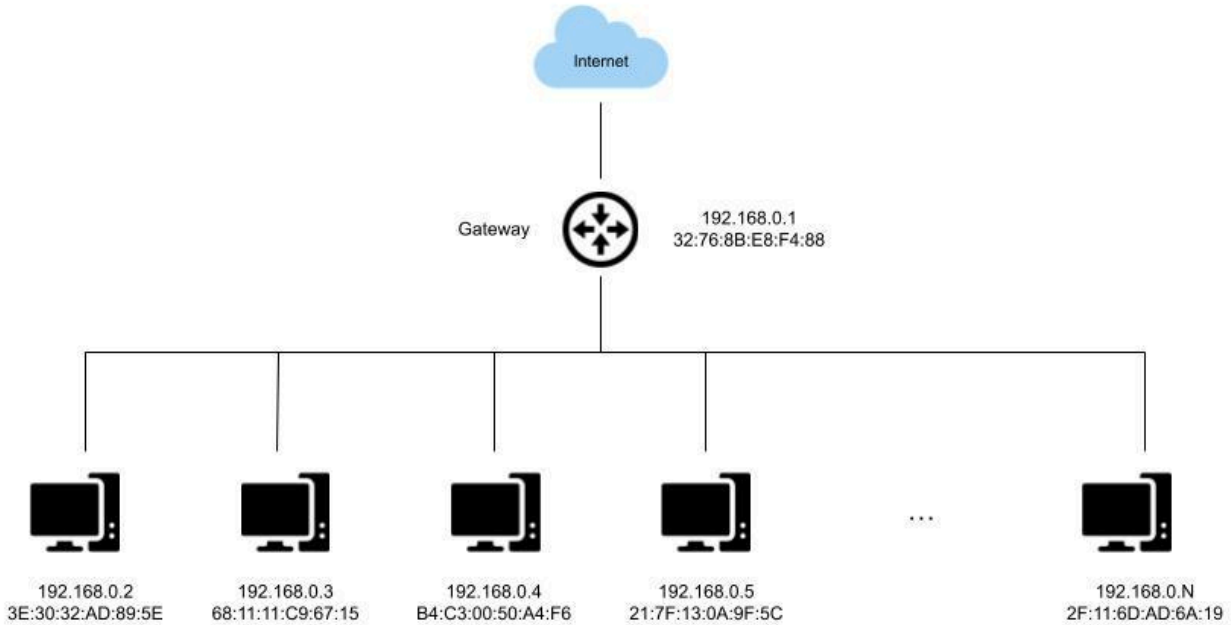
Exercise 5 - Network Security (6 points)

A local cafe has purchased many high-end PCs that can be rented hourly by clients to play videogames. Due to the low prices of this service and to the currently high prices of high-end PC components, the cafe has several dozens of daily clients. Each client has a unique username and a password that can be used to access any of the PCs in the cafe, and that are created on their first visit upon identification.

Even if inserting sensitive and personal data on a public and shared computer is discouraged, some clients did it anyway. These clients have found unexpected expenses made with their credit cards and reported the fact to the owner of the cafe.

The owner immediately starts an internal investigation, which involves a careful analysis of the traffic logs. During this analysis, the system administrator identifies the following sequence of packets, which leads them to conclude that some kind of network attack has been performed.

Answer to the following questions, based on the network topology and on the sequence of packets shown below.



Source IP	Source MAC	Destination IP	Destination MAC	Packet Type
192.168.0.3	68:11:11:C9:67:15	192.168.0.1	32:76:8B:E8:F4:88	ICMP echo request
192.168.0.1	B4:C3:00:50:A4:F6	192.168.0.3	68:11:11:C9:67:15	ICMP request Gateway 192.168.0.4
192.168.0.1	32:76:8B:E8:F4:88	192.168.0.3	68:11:11:C9:67:15	ICMP request Gateway 192.168.0.1

192.168.0.1	B4:C3:00:50:A4:F6	192.168.0.3	68:11:11:C9:67:15	ICMP request Gateway 192.168.0.4
192.168.0.3	68:11:11:C9:67:15	164.124.65.5	B4:C3:00:50:A4:F6	HTTP request

1. [3 points] What kind of attack can be recognized from this sequence of packets? What is the goal of the attacker? What steps are required to perform the attack?

This is an ICMP Redirect attack. The goal of the attacker is (in this specific attack instance) to redirect the traffic of the victim to its machine (MITM). The attacker needs to send an ICMP Redirect message to the victim, spoofing the IP of the gateway, specifying their own IP address as gateway. This way, the default gateway entry for that route will become the attacker's IP.

2. [1 point] What are the IP addresses of the attacker and of the victim?

The ICMP Redirect message indicates 192.168.0.4 as gateway: since the purpose of the attacker is to perform a MITM attack, this IP address either belongs to the attacker's machine or to another machine they control.

The victim is clearly the machine with 192.168.0.3 as IP address, as they are the receiver of the ICMP Redirect message.

3. [2 points] Is it possible to detect this attack while it is happening? If yes, how?

In general, detecting and mitigating ICMP Redirect attacks is non-trivial, as an ICMP Redirect message could be sent by a gateway as response to any of the packets sent by the victim. An easy option to completely avoid ICMP Redirect attacks is to completely disable ICMP Redirect in the OS, trading off performance as it would imply the risk of sending packets over non-optimal routes.

*However, this specific instance of ICMP Redirect attack is easily detectable, as the victim receives several ICMP Redirect messages following a **single** packet sent. In order to mitigate the attack, the victim could either completely disable ICMP Redirect, or apply a more fine-grained control mechanism: if multiple ICMP Redirect messages are received following a single packet sent, drop them both as it is a possible attempt of ICMP Redirect attack.*

Exercise 6 - Malware Security (6 points)

This exercise is structured from the perspective of a malware author. You will be asked to solve some issues that an attacker may face when developing a malicious program.

1. [2 points] Your program has to download some information from a server. You are confident that the IP address associated with that machine is known to AV providers. Therefore, you expect the AV deployed on the target machine to block your program as soon as it detects that address within the program code. Unfortunately, you don't have enough time to deploy a different server or route the malware traffic through proxies, VPNs, or analogue architectures. How would you **obfuscate** only the IP address to prevent an AV to statically identify it? Provide a simple example to explain your solution.

Different solutions possible:

1. Encrypt and decrypt the IP address (eg, you xor it with 9.7.6.5 and decrypt it at runtime)
2. Compute/recover it from a different IP (eg, you do $7.15.6.8 = 2.2.1.7 + 5.13.5.1$)
3. Encode/decode it in a different base (eg you rewrite it in base 16 and decode it back at runtime)

2. [2 points] You are now afraid that, having used a known malware generator downloaded from the Internet, other parts of your code will be detected statically by the AV. Thus, you decide to further obfuscate the entire malware by compressing it with UPX, a known code packer that compresses the payload and decompresses it at runtime. Can you identify any shortcoming in this strategy? Name **two** possible ways in which the AV could still detect your program, explaining how they relate to the use of UPX.

1. UPX is a well-known packer; hence, the AV can still easily obtain the original payload
2. As the code is packed/compressed, the AV may decide to dynamically analyze it, which may further expose the malicious nature of the program
3. Someone may claim that the use of a code compressor/packer is per se suspicious. Therefore, an AV may block it or signal its potential dangerousness

3. [2 points] You decide not to use UPX for the reasons you highlighted above. Furthermore, you can't use the obfuscation techniques you implemented at Q1 because you don't have enough time to reverse-engineer and modify the malware.

Luckily, you remember that you developed two obfuscation engines some time ago, both that can be easily and quickly applied to an existing binary executable. The first one is a polymorphic engine that encrypts the payload with a cryptographically-secure algorithm and can also generate a different decryption routine every time the malware propagates. The other one is a metamorphic engine that can generate different versions of the same malware at each propagation, by rearranging its control-flow graph, substituting instructions with equivalent ones, injecting dumb instructions to break possible signatures, and so on.

Considering your belief that your malware may be detected due to the hard-coded IP address or some invariant code fragments produced by the generator that you used, which of your two obfuscation engines would you use to carry out the attack? Motivate your choice, naming every assumption you need to make.

Polymorphic engine:

It is probably the safest option because it will definitely obfuscate the IP address when encrypting the payload. However, if the malware is supposed to infect more computers, this will probably result in a weaker strategy in the long run, as the real payload remains unaltered after decryption and, therefore, can be more easily detected at runtime.

Metamorphic engine:

If the engine does not obfuscate data or constant variables, then it will not be effective in evading the static AV as the IP address will still appear within the code. We therefore need to assume that the engine will somehow obfuscate the IP address for the attack to be successful.